

SELENIUM PYTHON AUTOMATION FRAMEWORK

Capstone Assignment

Submitted By-

Name: Ananya Saha

Enrollment No: 12023002001198

Stream: CSE

Institute: Institute Of Engineering &
Management, Kolkata

INDEX

SL. NO.	CONTENTS	PAGE NO.
1	Introduction	3
2	Project Objectives	3
3	Application Used	3
4	Tools and Technologies Used	4
5	Project Structure	4
6	Description of Folders and Files	5
7	Page Object Model	6
8	Configuration and Utility Classes	7
9	Test Data	7
10	Login Automation	8
11	Product Search Automation	8
12	PyTest	9
13	Unittest	9
14	Conftest.py	9
15	Screenshot on Failure	10
16	HTML Reporting	10
17	Test Execution	11
18	Test Execution Summary	12
19	Result	12
20	Conclusion	13
21	Future Scope	13
22	References	13

1. Introduction

Automation testing is used to test an application automatically instead of performing every test manually.

In this project, I have developed a Selenium automation framework using Python. The project is based on an e-commerce website, **TutorialsNinja Demo**.

The main functions automated in this project are:

- Login
- Product Search

The framework uses **PyTest** and **Unittest**. **Page Object Model** is used to keep the page-related code separate from the test cases. Test data is stored in **CSV** files. The project also includes configuration management, screenshots for failed tests and **HTML** test reports.

2. Objective of the Project

The main objective of this project is to create a simple and reusable Selenium Python automation framework.

The objectives are:

1. To automate the login functionality.
2. To automate product search functionality.
3. To use Selenium WebDriver with Python.
4. To use PyTest for executing test cases.
5. To use Unittest as part of the framework.
6. To implement Page Object Model.
7. To store test data separately in CSV files.
8. To use a configuration file for application and browser details.
9. To capture screenshots when a test fails.
10. To generate an HTML report after test execution.

3. Application Used

For this project, I used the **TutorialsNinja Demo E-Commerce Website**.
Website:

<https://tutorialsninja.com/demo/>

The website provides different e-commerce functions such as login, product search, product details and shopping cart.

For this assignment, I focused mainly on the Login and Product Search functions.

4. Software and Technologies Used

The following tools and technologies were used:

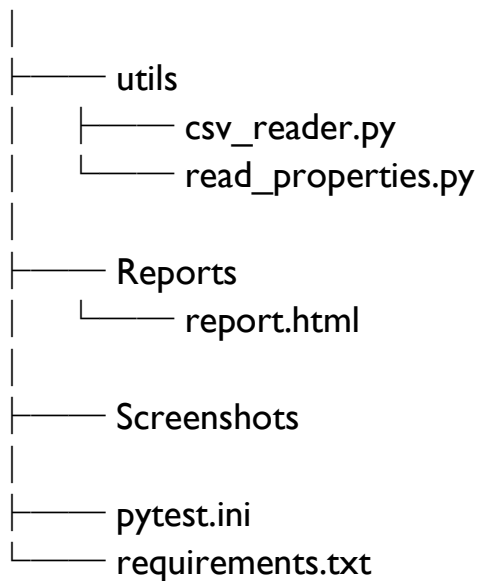
TECHNOLOGY	PURPOSE
Python	Programming language
Selenium WebDriver	Browser automation
PyTest	Test execution
Unittest	Additional test framework
Page Object Model	Organizing page-related code
CSV	Storing test data
pytest-html	Generating HTML report
VS Code	Writing and running the project
Google Chrome	Browser used for testing

5. Project Structure

The project is divided into different folders so that the files are easy to manage.

Assignment

```
|
|----- config
|       |----- config.ini
|
|----- pages
|       |----- base_page.py
|       |----- login_page.py
|       |----- search_page.py
|
|----- tests
|       |----- conftest.py
|       |----- test_login.py
|       |----- test_search.py
|       |----- test_unittest.py
|
|----- testdata
|       |----- logindata.csv
|       |----- searchdata.csv
```



6. Description of the Folders and Files

6.1 Config Folder

The config folder contains:

`config.ini`

This file contains the application URL and browser information.

Example:

`baseURL = https://tutorialsninja.com/demo/`

`browser = chrome`

The advantage of keeping these details in a separate file is that I don't have to write the URL directly in every test case.

6.2 Pages Folder

The pages folder contains the Page Object Model classes.

It contains:

`base_page.py`

`login_page.py`

`search_page.py`

base_page.py

This file contains common Selenium functions.

For example, functions for:

- Clicking an element
- Entering text
- Getting text
- Checking whether an element is visible
- Waiting for elements

These functions can be reused by different page classes

login_page.py

This file contains the locators and functions related to the login page.

Some of the functions used are:

```
click_my_account()
click_login_link()
set_email()
set_password()
click_login()
```

search_page.py

This file contains the functions required for product searching.

It handles:

- Entering the search text
- Clicking the search button
- Checking the search result
- Checking the message when no product is found

7. Page Object Model

Page Object Model, also called POM, is used in this project to separate the test cases from the actual page operations.

For example, instead of writing Selenium locators directly inside the login test, I create a LoginPage object.

Example:

```
lp = LoginPage(self.driver)
lp.click_my_account()
lp.click_login_link()
lp.set_email(username)
lp.set_password(password)
lp.click_login()
```

This makes the test case easier to read.

If a locator changes later, it can be changed inside login_page.py instead of changing every test case.

8. Utils Folder

The utils folder contains reusable utility files.

It contains:

```
csv_reader.py  
read_properties.py
```

csv_reader.py

This file is used to read the test data from CSV files.

The test cases can then use the data without writing it directly inside the Python code.

read_properties.py

This file reads the values from config.ini.

It is used to get information such as:

- Application URL
- Browser name

9. Test Data

The test data is stored separately in the testdata folder.

There are two CSV files:

```
logindata.csv  
searchdata.csv
```

Login Data

The login CSV contains the username, password and expected result.

Example:

```
username,password,expected  
testuser1@example.com,testpass1,Fail  
invaliduser@example.com,wrongpass,Fail
```

These are negative login test cases.

Search Data

The search CSV contains different search inputs.

Examples used in the project include:

```
MacBook  
iPhone  
InvalidProductxyz
```

The first two are valid product searches, while the last one is used to check the no-product result.

10. Login Automation

The login test is present in:

tests/test_login.py

PyTest parameterization is used for the login data.

The test reads the data from logindata.csv and runs the same test for different username and password combinations.

The basic flow is:

Open website
↓
Click My Account
↓
Click Login
↓
Enter email
↓
Enter password
↓
Click Login
↓
Check the result

For invalid login data, the test checks that login does not succeed and the expected warning message is displayed.

11. Product Search Automation

The product search test is present in:

tests/test_search.py

The test data is read from searchdata.csv.

The basic flow is:

Open website
↓
Enter product name
↓
Click Search
↓
Check search result

The test includes:

- MacBook search
- iPhone search
- Invalid product search

For the invalid product, the test checks that the appropriate no-product message is displayed.

12. PyTest

PyTest is the main test framework used in this project.

It is used for:

- Running the test cases
- Fixtures
- Parameterization
- Assertions
- Generating the test report

For example:

```
@pytest.mark.parametrize(
    "username,password,expected",
    read_data_from_csv("logindata.csv")
)
```

This allows the same test to run using multiple rows from the CSV file.

13. Unittest

The assignment also requires Unittest.

For this purpose, I added:

```
tests/test_unittest.py
```

The file uses Python's Unittest framework.

For example:

```
import unittest
class TestLoginUnittest(unittest.TestCase):
```

The test checks whether the login page is opened correctly.

The browser is started before the test and closed after the test.

The Unittest test is also executed successfully along with the PyTest tests.

14. Conftest.py

The conftest.py file contains the common PyTest setup.

It is responsible for creating the browser driver.

The browser is selected using the configuration file.

The fixture performs actions such as:

- Starting Chrome
- Maximizing the browser
- Setting implicit wait
- Providing the driver to the test
- Closing the browser after execution

The file also contains the screenshot handling used when a test fails.

15. Screenshot on Failure

Screenshot functionality has been added to the framework.

When a test fails, the framework tries to capture a screenshot of the browser.

The screenshots are stored in:

Screenshots

The screenshot can also be attached to the HTML report.

This is useful because it helps to understand what was visible in the browser when the test failed.

16. HTML Reporting

The project uses pytest-html to generate the test execution report.

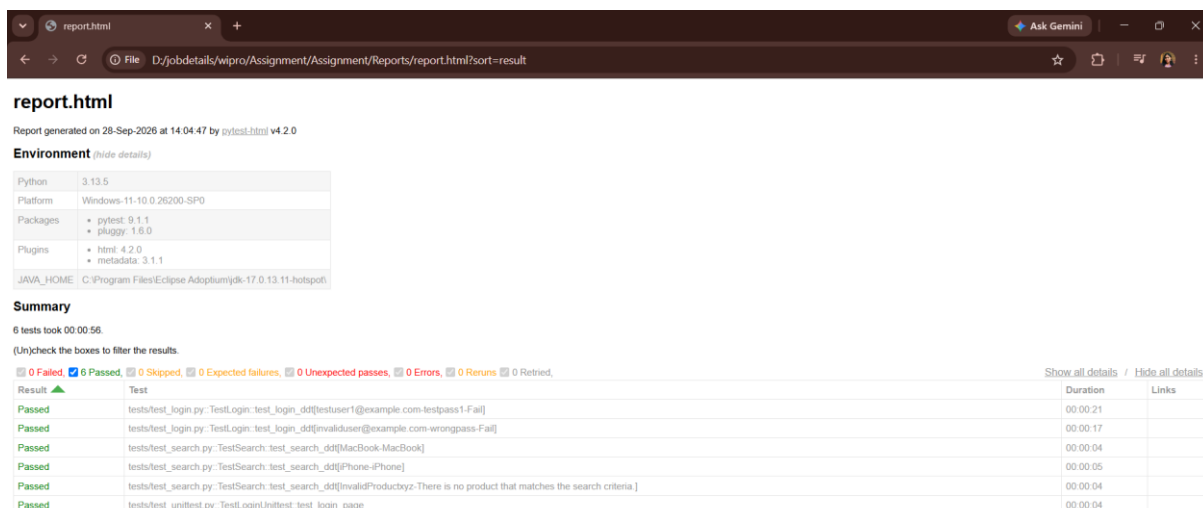
The report is generated at:

Reports/report.html

The report contains:

- Number of tests
- Passed tests
- Failed tests
- Skipped tests
- Individual test results
- Execution time

The report makes it easier to see the final result without checking the terminal output.



17. Test Execution

The complete test suite was executed from the project directory using:

```
python -m pytest
```

The following test files were executed:

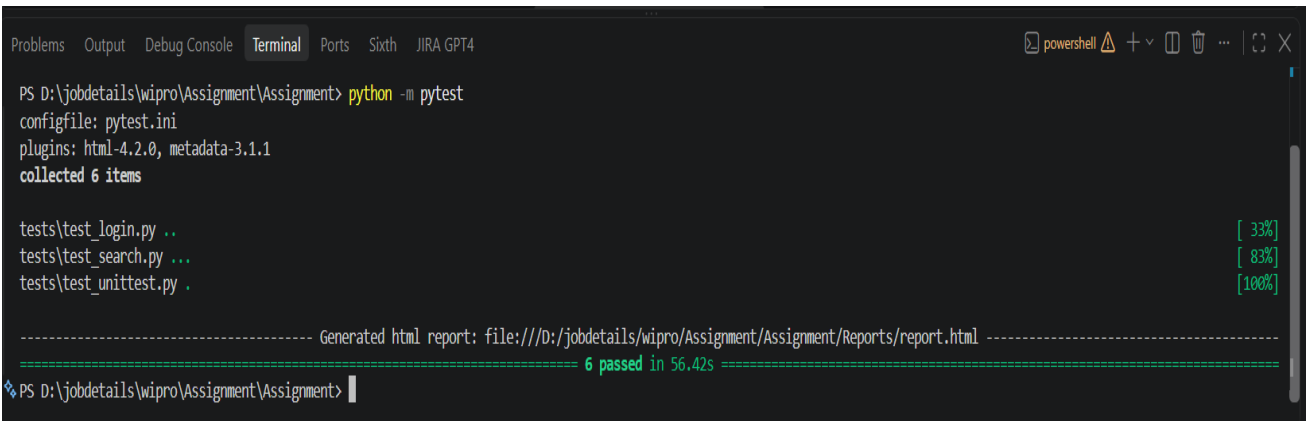
```
tests/test_login.py
tests/test_search.py
tests/test_unittest.py
```

The final execution result was:

6 passed in 52.64s

The result was:

Result	Number
Total Tests	6
Passed	6
Failed	0
Skipped	0
Errors	0



```
PS D:\jobdetails\wipro\Assignment\Assignment> python -m pytest
configfile: pytest.ini
plugins: html-4.2.0, metadata-3.1.1
collected 6 items

tests/test_login.py .. [ 33%]
tests/test_search.py ... [ 83%]
tests/test_unittest.py . [100%]

----- Generated html report: file:///D:/jobdetails/wipro/Assignment/Assignment/Reports/report.html -----
===== 6 passed in 56.42s =====
PS D:\jobdetails\wipro\Assignment\Assignment>
```

18. Test Execution Summary

The six tests consisted of:

Login Tests

2 tests

- Invalid login test 1
- Invalid login test 2

Search Tests

3 tests

- MacBook search
- iPhone search
- Invalid product search

Unittest

1 test

- Login page validation

Therefore:

2 Login Tests

+

3 Search Tests

+

1 Unittest

6 Tests

All six tests passed successfully.

19. Result

The automation framework was successfully executed.

The final result was:

6 passed

0 failed

0 skipped

0 errors

The HTML report was also generated successfully.

This confirms that the implemented login, product search and Unittest scenarios are working with the current test data and application.

20. Conclusion

In this project, I created a Selenium automation framework using Python. I used PyTest for the main test execution and also included Unittest as required. Page Object Model was used for the login and search pages. The test data was kept separately in CSV files and configuration details were kept in a configuration file. The framework also includes common browser setup, screenshot handling and HTML reporting. The complete test suite was executed successfully and all six tests passed.

21. Future Scope

The framework can be extended further by adding more e-commerce scenarios such as:

- Add product to cart
- Update product quantity
- Remove product from cart
- Verify cart details
- Checkout process
- Logout

Other improvements can include:

- Excel or JSON test data
- More browser support
- Jenkins integration
- More test cases
- Better logging

22. References

1. TutorialsNinja Demo E-Commerce Website
<https://tutorialsninja.com/demo/>
2. Selenium WebDriver
3. Python PyTest
4. Python Unittest
5. pytest-html